CS471 Project Proposal: PolyRack-Beyond Hotness

Tiered Memory Management Beyond Hotness

Mayeul Besnard
EPFL
Lausanne, Switzerland
mayeul.besnard@epfl.ch

Sergio Boffi
EPFL
Lausanne, Switzerland
sergio.boffi@epfl.ch

***Abstract*—Tiered memory systems combine a fast tier (local DRAM) with a larger but slower tier (e.g., CXL-attached memory) and typically decide data placement based on access frequency ("hotness"). Liu et al. [1] argue that hot data is not necessarily performance-critical, because memory-level parallelism (MLP) can hide most of the slow-tier latency of some accesses but not of others. They introduce the Amortized Offcore Latency (AOL) metric and build two mechanisms on top of it: SOAR, an AOL-guided first-placement policy, and ALTO, an AOL-based filter for page migrations. In this project we will reproduce the main claims of the paper (hotness is a poor proxy of performance, AOL is a better predictor of slowdown, SOAR and ALTO outperform state-of-the-art tiering systems) on a traditional server and, if time permits, evaluate whether these techniques also improve memory management on the PolyRack disaggregated rack architecture [2].**

I. Introduction and Problem Statement

A. Why is the problem important?

Server performance can be measured in many different ways, but the most common and relevant remains the number of instructions executed per cycle: Perf $\propto$ IPC $= 1/\text{CPI}$. Whenever the CPU requests data from memory, an LLC (Last-Level Cache) miss may occur, meaning that the requested piece of data is not available in any of the cache levels. When this happens, the CPU might wait many cycles to get an answer from the main memory (stall cycles), and therefore performance starts degrading. Memory-intensive datacenter workloads keep growing in footprint, and provisioning all of their memory in DRAM is expensive. Tiered memory architectures add a cheaper, higher-capacity slow tier (CXL memory, remote NUMA node) next to the fast DRAM tier. Since the new tier is slower (2–3$\times$ slower than DRAM), the way data is distributed across tiers determines how many memory stalls the application suffers.

B. What is involved in the problem?

The problem involves:

- The two memory tiers and their latencies T_{DRAM} and T_{CXL}
- The access pattern of each object
- Hot and cold objects/pages
- The initial placement policy
- The page migration policy

A simple first-order model makes the dependencies explicit. Let α be the fraction of memory accesses served by DRAM. The average memory latency is

$$T_{\text{total}} = \alpha\, T_{\text{DRAM}} + (1-\alpha)\, T_{\text{CXL}}. \tag{1}$$

Let $\beta_{\text{DRAM,hot}}$ ($\beta_{\text{CXL,hot}}$) be the fraction of DRAM (CXL) accesses that target hot data. Splitting the accesses to each tier into hot and cold data gives

$$\begin{aligned} T_{\text{total}} = {} & \underbrace{\alpha\, \beta_{\text{DRAM,hot}} T_{\text{DRAM}}}_{\text{accesses to hot data in DRAM}} + \underbrace{\alpha\, (1-\beta_{\text{DRAM,hot}}) T_{\text{DRAM}}}_{\text{accesses to cold data in DRAM}} \\ & + \underbrace{(1-\alpha)\, \beta_{\text{CXL,hot}} T_{\text{CXL}}}_{\text{accesses to hot data in CXL}} + \underbrace{(1-\alpha)(1-\beta_{\text{CXL,hot}}) T_{\text{CXL}}}_{\text{accesses to cold data in CXL}}. \end{aligned} \tag{2}$$

SOTA tiering systems put hot data into DRAM, and following this model, this means that we decrease $(1-\alpha)\, \beta_{\text{CXL,hot}} T_{\text{CXL}}$ to increase $\alpha\, \beta_{\text{DRAM,hot}} T_{\text{DRAM}}$. Since we know that CXL accesses are more costly in terms of latency, this might look like an optimal solution, but the next section shows why this tiering system doesn't always help lower T_{total}.

C. What causes the problem?

First, we need to point out that slow-tier accesses and page migrations are expensive in terms of latency (each migration can take several μs). To reduce the slowdown, we need to find a way to decide whether a page should migrate or whether an object should be placed in the fast or the slow tier. Having an effective initial placement algorithm allows us to have fewer page migrations, hence reducing the total slowdown. As we have seen, SOTA uses hotness as a metric to make tiering decisions, but hotness ignores memory-level parallelism (MLP). In out-of-order (OoO) cores, memory requests can be overlapped (when possible), so the effective cost of an access is the latency of that access divided by the number of concurrent accesses. This breaks the assumption of the model above, where every access pays the full latency of its tier.

Let us consider two different data structures of the same size that need to be repeatedly scanned: an array and a linked list. When scanning an array, we already know the address of each element of the array, but in a linked list, we need to read element x to be able to reach element x+1. Suppose we perform this scan on a tiered memory system with an OoO processor: the accesses to the array can be parallelized

($MLP \geq 1$), while the accesses to the linked list must be serialized ($MLP = 1$). Let us suppose that hotness is measured over a fixed period of time. We can now imagine that the array has been accessed many times, making it hotter, so it gets placed in the fast tier. This decision might help us reach the array faster, but it drastically increases the time needed to access the linked list. Swapping the positions leads us to a lower total slowdown.

Hotness therefore measures how *often* data is accessed, but not how much each access *costs*: it is an unfaithful proxy of performance.

D. Does prior work solve the problem?

No. SOTA tiering systems such as TPP, Nomad, Linux NUMA Balancing Tiering (NBT) and Colloid all rely on hotness. On the paper's microbenchmark (one sequential, high-MLP "hot" thread and one pointer-chasing, low-MLP "cold" thread), they underperform even a naive *Cold-on-DRAM* placement, and fall behind first-touch allocation without migrations (NoTier) by 12–14% [1, Fig. 1]. On real workloads such as `bc-urand` (GAPBS betweenness centrality on a uniform random graph), they can be up to 217% slower than NoTier [1, Fig. 4].

E. Problem definition, scope and success criteria

Problem. *Reduce the slowdown of memory-intensive workloads on tiered memory systems, by reproducing and validating the AOL-based approach of on our hardware and, if time permits, on PolyRack.*

Scope.

- *Infrastructure:* a two-tier server (local DRAM plus a CXL expander, or a remote NUMA node emulating it); PolyRack only as an extension.
- *Inputs:* the hot/cold microbenchmark of [1] and GAPBS graph workloads (starting from `bc-urand`), run with 10–90% of their RSS in the slow tier.
- *What we can change:* software only, i.e., the initial placement of objects; the baselines' page-migration policies are used as they are. Hardware, workload algorithms and their inputs are fixed.

Success criteria. The metric is the execution-time slowdown with respect to All-on-DRAM (lower is better), plus the Pearson correlation between predicted and measured slowdown for the metric question. Baselines are NoTier (first-touch, no migrations) and the latest hotness-based systems TPP, NBT, Nomad and Colloid, all run with the same fast-tier capacity. Since our hardware differs from the paper's, the reproduction is successful if the *qualitative* trends hold (ranking of the policies and sign of the improvements), not the exact numbers.

F. Overview of the proposed solution [1]

AOL. The paper first shows (via an offline study of 56 workloads) that the ratio of LLC stall cycles to total cycles, $P = s_{\text{LLC}}/c$, is correlated with the slowdown on the slow tier, but overestimates it for high-MLP workloads. It then defines

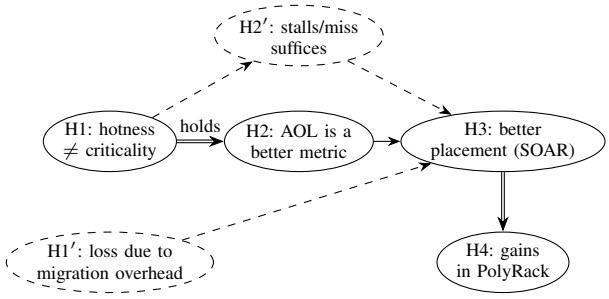


Fig. 1. Hypothesis tree. Solid: main hypotheses from [1]; dashed: alternatives we test in parallel. If H1′ holds instead of H1, the problem is migration overhead: we skip Q2 and evaluate SOAR directly, since it avoids most migrations.

AOL = Latency/MLP, computed from offcore performance counters, and models the slowdown as $S = P \times K$ with a correction factor $K = f(\text{AOL}) = 1/(a + b/\text{AOL})$. The constants a, b depend only on the hardware and can be calibrated with microbenchmarks; the Pearson correlation between predicted and measured slowdown improves from 0.869 to 0.951.

SOAR (Static Object Allocation based on Ranking) profiles an application, ranks objects by their AOL-weighted contribution to performance, and places the highest-ranked objects in DRAM at allocation time. This yields near-optimal first placement and removes the need for most later migrations. The paper also proposes ALTO, an AOL-based filter for page migrations; we leave it out of scope to focus on placement.

G. Project goals

With this project we aim to:

- reproduce the main results of "Tiered Memory Management Beyond Hotness" [1]: the failure of hotness-based metrics, the AOL-based slowdown predictor, and the gains of SOAR over SOTA tiering systems;
- if time permits, test AOL and SOAR on the PolyRack architecture [2] and quantify their benefit there.

II. Questions, Hypotheses and Tests

The overarching question is: *how can we improve server performance by reducing effective memory latency, using metrics better than hotness and building better data-placement procedures on top of them?* We break it down into the following prioritized questions. For each question we give an insight, a hypothesis, the result that would refute it, and the test; where the evidence could point elsewhere we also state an alternative hypothesis (primed). Questions are ordered by dependency (Fig. 1): a child hypothesis is tested only once its parent has been confirmed.

A. Q1: Are hotness metrics really informative?

- **Insight (I1):** out-of-order cores overlap independent misses, so high-MLP (often hot) accesses hide most of the slow-tier latency, while low-MLP accesses pay all of it.

- **Hypothesis (H1):** with the same DRAM capacity, placing the hot, high-MLP data in DRAM yields a *higher* slowdown than placing the cold, low-MLP data in DRAM.
- **Refuted if:** on the microbenchmark, Hot-on-DRAM has a slowdown lower than or equal to Cold-on-DRAM.
- **Test (E1):** reproduce the hot/cold microbenchmark (sequential vs. pointer-chasing threads, cf. Fig. 1 of [1]) with *static* placements (All-on-DRAM, Hot-on-DRAM, Cold-on-DRAM) and migrations disabled, so that the result depends only on placement and not on migration cost.
- **Alternative (H1′):** SOTA systems lose because of the cost of their migrations, not because they mis-place data. *Test (E1′):* run NoTier and the SOTA systems on the same benchmark and record the number of migrated pages. If the static placements perform alike but SOTA is slower than NoTier, H1′ holds.

B. Q2: If not, how can a better metric be defined?

- **Insight (I2):** the average stall cycles per LLC miss vary by up to 4× across workloads [1]: MLP amortizes the latency, so a predictor that ignores it (LLC stalls/cycle, P) overestimates the slowdown of high-MLP workloads.
- **Hypothesis (H2):** the AOL-corrected predictor $P \times K$, with $K = f(\text{AOL})$ fitted empirically on our machine, correlates better with the measured slowdown than P alone and than hotness (sampled access counts).
- **Refuted if:** the Pearson coefficient of $P \times K$ is not higher than that of P (or of hotness) on our set of workloads.
- **Test (E2):** for each workload, measure the slowdown on the slow tier together with the counters for P, AOL and access counts; calibrate a, b of K with the sequential/pointer-chasing microbenchmarks; compare the Pearson coefficients of the three predictors (cf. Fig. 2 of [1]).
- **Alternative (H2′):** a simpler MLP-aware metric, the average LLC stalls per LLC miss without a fitted K, predicts the slowdown as well as $P \times K$. It is evaluated in the same experiment as E2 at no additional cost.

C. Q3: With this metric, can we improve first data placement (SOAR)?

- **Insight (I3):** if the metric identifies performance-critical objects, placing them in DRAM at allocation time avoids both mis-placement and later migrations.
- **Hypothesis (H3):** SOAR achieves a lower slowdown than NoTier and every SOTA system at the same fast-tier capacity.
- **Refuted if:** NoTier or a SOTA system has an equal or lower slowdown (beyond run-to-run variance) in the majority of the (workload, slow-tier ratio) configurations.
- **Test (E3):** compare the slowdown of SOAR vs. NoTier, TPP, NBT, Nomad and Colloid on the selected workloads, at 10–90% slow-tier ratios (cf. Fig. 4 and Table 4 of [1]).

D. Q4: Can these elements improve PolyRack?

- **Insight (I4):** PolyRack disaggregates memory into dedicated memory servers reached through a CXL-class fabric, which creates a fast/slow memory hierarchy similar to a tiered server.
- **Hypothesis (H4):** AOL-based placement (SOAR) reduces the slowdown on PolyRack with respect to its default memory management.
- **Refuted if:** no reduction beyond run-to-run variance, after recalibrating K for PolyRack.
- **Test (E4):** run the same workloads on PolyRack with and without SOAR, and compare against the traditional tiered server. Tested only if H3 holds.

E. Handling failed hypotheses

A refuted hypothesis is an anomaly that we analyze before moving on:

- *H1 fails:* our slow tier may hide latency differently (e.g., lower latency ratio, more effective prefetching). We sweep the slow-tier latency (uncore frequency) to find whether the effect appears; if H1′ holds instead, we skip Q2 and evaluate SOAR directly, since it avoids most migrations.
- *H2 fails:* we check whether K fitted on our hardware has the same shape as in the paper, and use H2′ (if it holds) as the metric driving SOAR.
- *H3 fails:* we analyze per workload which objects SOAR ranks incorrectly, and test whether recalibrating K for our platform fixes the gap.

III. Evaluation Methodology

A. Experimental setup

We will first reproduce the results on a traditional server, and only then move to PolyRack. Following [1], if no real CXL device is available the slow tier will be emulated with a remote NUMA node (lowering its uncore frequency and disabling its cores, which in the paper yields 90/190 ns fast/slow latencies). We will use the open-source implementation of SOAR (https://github.com/MoatLab/SoarAlto), Linux `perf` and the offcore/LLC performance counters needed to compute P and AOL, and `numactl`/cgroups to control the fast/slow-tier ratio. [TODO: Confirm server access, CPU/kernel version, CXL vs. NUMA emulation, and whether experiments run on bare metal or in a VM.]

B. Workloads

We will start from `bc-urand` and other graph workloads from the GAP Benchmark Suite (GAPBS), which stress tiered memory with irregular accesses, plus the hot/cold microbenchmark of Q1. Time permitting, we will add workloads from the other categories used in the paper (machine learning, caching, HPC). [TODO: Agree on the final workload list with the TA (and on PolyRack's home workloads).]

C. Baselines and metrics

Baselines: All-on-DRAM (upper bound), NoTier (first-touch, no migrations), and the hotness-based SOTA systems TPP, NBT, Nomad and Colloid. Metrics: slowdown relative

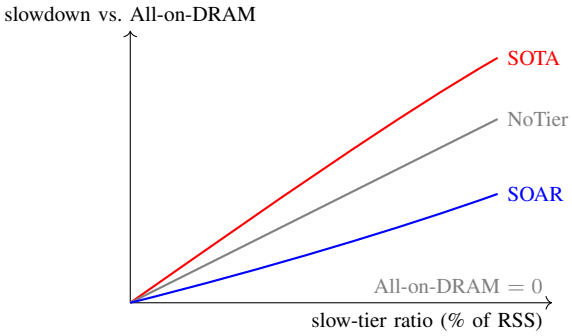


Fig. 2. Sketch of the expected main result. All-on-DRAM is the zero line; NoTier is first-touch without migrations. AOL-based placement (SOAR) keeps the slowdown lower than hotness-based SOTA and NoTier across slow-tier ratios.

to All-on-DRAM (lower is better), Pearson correlation between predicted and measured slowdown, AOL and average memory latency, and migration traffic (number of page promotions/demotions), all measured across several fast/slow-tier ratios (e.g., 10–90% of the workload's RSS in the slow tier). Every configuration is run at least three times; we report mean and standard deviation, and consider two policies different only when their means differ by more than the run-to-run standard deviation.

IV. Final Deliverable

A. Concrete goals

By the end of the semester we want to (1) have a working tiered-memory setup with the baselines and SOAR running, (2) confirm or refute H1–H3 (and the alternatives H1′, H2′) on our hardware, and (3) if time permits, provide a first evaluation of H4 on PolyRack.

B. Expected results

Our key figure (Fig. 2) will plot the slowdown w.r.t. All-on-DRAM (Y-axis) as a function of the fraction of the working set placed in the slow tier (X-axis), for NoTier, a hotness-based SOTA system and SOAR. We expect slowdown to grow with the slow-tier ratio for all policies, with SOTA sometimes worse than NoTier (due to useless migrations) and SOAR consistently below both. We will also show the predicted-vs.-measured slowdown scatter plot for P, $P \times K$, hotness and stalls/miss (Q2).

V. Plan and Milestones

Each milestone lists its deadline and the follow-up question it answers.

1) **Oct. 13:** proposal and presentation. *Follow-up:* which servers, CXL device or NUMA emulation, and workloads are available to us (Sec. VI)?
2) **Oct. 23:** environment ready: tiered (or NUMA-emulated) memory, SOAR code and baselines built, counters for P and AOL readable with `perf`. *Follow-up:* does our CPU expose the offcore counters AOL needs?
3) **Oct. 30:** E1 and E1′ done. *Follow-up:* does H1 hold on our hardware, or H1′?
4) **Nov. 12 (status report):** E2 done (K calibrated, Pearson coefficients of P, $P \times K$, hotness and H2′); first SOAR results on `bc-urand`. *Follow-up:* which metric should drive SOAR on our platform?
5) **Nov. 27:** E3 done on all selected workloads and slow-tier ratios. *Follow-up:* on which configurations does SOAR lose, and why?
6) **Dec. 4:** (if time permits) E4 on PolyRack. *Follow-up:* can AOL be measured and calibrated on PolyRack's memory path?
7) **Dec. 10 / Dec. 17 / Dec. 20:** poster, poster session and final report.

VI. Open Questions for the Teaching Team

1) Will we test AOL/SOAR directly on PolyRack, or on a traditional server beforehand? (Our plan assumes the latter.)
2) Which home workloads will we work with?
3) How does access to the servers work?
4) How will we set up our experiments (programming language, bare metal vs. VM, etc.)?

References

[1] J. Liu, H. Hadian, H. Xu, and H. Li, "Tiered Memory Management Beyond Hotness," in *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, Boston, MA, USA, Jul. 2025.

[2] P. Poorsarvi Tehrani, S. Lin, A. Ning, B. Falsafi, D. Atienza, T. Bourgeat, and S. Kashyap, "PolyRack: Servers for the Post-Moore Era," EPFL, 2026.